

AparcAR

Manual de arquitectura para quien nunca vio este codigo

De punta a punta: que hace el sistema, como esta organizado, que tecnologia usa cada pieza y por que.

12 capitulos	43 tecnologias explicadas	28 endpoints documentados	226 tests del proyecto
------------------------	--	--	-------------------------------------

Backend: Java 21 + Spring Boot 4 + PostgreSQL + Docker

Frontend: Next.js 16 + React 19 + Tailwind CSS 4

Como usar este manual

Este documento asume **cero conocimiento previo** del proyecto y muy poco de programacion. Cada termino tecnico se explica la primera vez que aparece. Si algo no se entiende, probablemente este explicado unas paginas antes, no despues.

Hay tres formas de leerlo, segun para que lo necesites:

Si quieres...	Lee...	Tiempo
Entender el proyecto por primera vez	Todo, en orden, de principio a fin	Una tarde
Saber que hace una tecnologia puntual	Solo el capitulo 3 , buscando la entrada que te interesa	5 minutos
Poder tocar el codigo maniana	Capitulos 1, 2, 4, 5 y la chuleta del 12	Una hora
Prepararte para que te pregunten	La autoevaluacion del 12.3 , y volver a los capitulos que no sepas contestar	Depende de vos

Convenciones visuales

Este tipo de letra = un archivo, un comando o algo que se escribe literal.

Los recuadros **celestes** agregan contexto. Los **verdes** marcan un concepto clave que conviene memorizar. Los **amarillos** son advertencias: cosas que confunden a todo el mundo. Los **rojos** son problemas reales que el proyecto tuvo.

Un consejo de estudio

No intentes memorizar los nombres de los 130 archivos. Memoriza **las cuatro capas** (controller, service, repository, entity) y **que hace cada una**. Con eso podes deducir donde esta cualquier cosa, que es mucho mas util que recordar una lista.

Indice

1. Que es AparcAR y que problema resuelve

- El problema del mundo real
- El vocabulario del dominio: los 5 conceptos
- Las reglas de negocio: lo que el sistema NO deja hacer

2. La arquitectura en una servilleta

- Las tres capas y la analogia del restaurante
- Que pasa cuando hacés click en "Confirmar reserva": los 11 pasos

3. Diccionario de tecnologías (43 entradas)

- La base del backend: Java, Spring Boot, Maven, anotaciones
- Persistencia: JPA, Hibernate, Liquibase, PostgreSQL, H2
- Seguridad: Spring Security, Basic, JWT, BCrypt, Bucket4j, Caffeine
- Utilidades: Lombok, validaciones, DTOs, Swagger, Actuator, JaCoCo
- Testing del backend: JUnit, Mockito, MockMvc
- El frontend: React, Next.js, Tailwind, Axios, Zustand, Zod
- Testing del frontend: Vitest, Testing Library, jsdom

4. El arbol de carpetas, comentado

- Vista general y las carpetas generadas que no van a git
- El backend por fuera y por dentro
- Configuración, perfiles y migraciones
- El frontend completo

5. De donde salen los endpoints del backend

- El mecanismo de las anotaciones, en tres pasos
- Los verbos HTTP
- Tabla completa de los 28 endpoints

6. Como funciona la seguridad, paso a paso

- Momento 1: el login (8 pasos)
- Momento 2: todos los demás pedidos
- 401 vs 403
- La tabla de permisos tal como esta escrita
- CSRF, stateless y CORS
- La protección del lado del frontend

7. La base de datos: como se crea, como se guarda

- Las tablas que existen hoy y quien crea cada una
- Liquibase vs Hibernate: por que conviven y que problema trae
- Que base se usa en cada situación

8. Docker: por que un contenedor y no un servidor normal

- Que es un contenedor y que problema resuelve
- Las siete razones concretas en este proyecto
- El Dockerfile y el truco del multi-stage build

9. Manual de arranque: como se ejecuta todo

- Que instalar una sola vez
- Levantar el backend y el frontend
- Correr los tests
- Tabla de problemas frecuentes y soluciones

10. Los tests: 226 razones para dormir tranquilo

- Caja blanca y caja negra
- Los tests que mas vale la pena leer
- Que es un test de regresion

11. Tres bugs reales que ensenian mas que cien teorias

- El login que pedia la contraseña dos veces
- El vehiculo recién cargado que no aparecia
- El usuario que no se podia borrar

12. Chuleta final y autoevaluacion

- Donde busco cada cosa
- Las diez frases que resumen todo
- 15 preguntas de autoevaluacion

Que es AparcAR y que problema resuelve

Antes de mirar una sola linea de codigo hay que entender **para que existe** este programa. Un sistema de software no es codigo: es una respuesta a un problema que alguien tiene en el mundo real. Si no entiendes el problema, el codigo te va a parecer una pila arbitraria de archivos.

1.1 El problema del mundo real

Una oficina, una universidad, un sanatorio o un organizador de eventos tiene un estacionamiento con una cantidad limitada de lugares. Hoy lo administran a ojo: planillas sueltas, un guardia anotando patentes a mano en un cuaderno, visitantes que llegan sin saber si van a tener lugar. El resultado es siempre el mismo: **sobreocupacion** (llegan mas autos que lugares), **demoras en el acceso** (la barrera se convierte en un cuello de botella) y **cero control real** de quien entro al predio.

1.2 La solucion que propone AparcAR

AparcAR es una aplicacion web donde el establecimiento carga sus cocheras, registra a los visitantes con sus vehiculos, y les asigna una reserva para una fecha concreta. El sistema garantiza que **nunca** se asignen dos reservas a la misma cochera el mismo dia, y que el tipo de vehiculo entre en el tipo de cochera que le tocó.

1.3 El vocabulario del dominio (esto es lo mas importante del capitulo)

Todo proyecto tiene un puñado de sustantivos que se repiten en todos lados: en los nombres de archivos, en las tablas de la base de datos, en las URLs. A eso se le llama el **dominio**. En AparcAR son cinco, y estan en castellano:

Concepto	Que representa	Datos que guarda
Visitante	Una persona fisica que va a estacionar en el predio. No necesariamente tiene cuenta de login: puede ser un invitado que el admin carga a mano.	nombre, documento (DNI, unico), telefono, email
Vehiculo	Un auto, moto o camion que pertenece a un visitante . Un visitante puede tener varios; un vehiculo pertenece a uno solo.	patente (unica, siempre en mayusculas), tipo (AUTO / MOTO / CARGA), visitante dueño
Cochera	Un lugar fisico de estacionamiento del predio.	numero (unico), sector, tipo (AUTO / MOTO / ACCESIBLE / CARGA), estado (habilitada o no)
Reserva	El cruce de todo lo anterior: este visitante, con este vehiculo, ocupa esta cochera, esta fecha.	fecha, visitante, vehiculo, cochera, estado (CONFIRMADA / CANCELADA)
AppUser	Una cuenta de login del personal interno . Es lo unico que tiene contraseña. Un AppUser puede opcionalmente estar vinculado a un Visitante (su propio perfil).	nombre, email, password encriptada, telefono, activo si/no, roles

La confusion clasica: Visitante no es lo mismo que Usuario

Un **Visitante** es "alguien que estaciona". Un **AppUser** es "alguien que se loguea al sistema". Son tablas distintas y conceptos distintos. Un guardia tiene AppUser pero quizas nunca estaciona; un invitado externo tiene Visitante pero no tiene cuenta.

La tabla visitantes tiene una columna app_user_id justamente para el caso en que una persona sea las dos cosas: un empleado que ademas estaciona su propio auto.

1.4 Las reglas de negocio (lo que el sistema NO deja hacer)

Un sistema se define tanto por lo que permite como por lo que prohíbe. Estas cinco reglas viven en el archivo `service/impl/ReservaService.java` y son el corazón del proyecto. Memorízalas:

1. **El vehículo tiene que ser del visitante.** No puedes reservar poniendo el auto de otro. Si el `visitanteId` y el `vehículoId` no coinciden, error 400.
2. **El tipo tiene que ser compatible.** Una moto no va en una cochera de tipo AUTO. El sistema compara `cochera.tipo` contra `vehículo.tipo`.
3. **Excepción: las cocheras ACCESIBLE aceptan cualquier vehículo.** Es la única excepción a la regla anterior, y está escrita explícitamente en el código.
4. **Una cochera, una reserva confirmada por día.** Si ya hay una reserva CONFIRMADA para esa cochera y esa fecha, se rechaza. Esto es lo que garantiza cero sobreocupación.
5. **No se reserva en el pasado.** La fecha tiene que ser hoy o después (anotación `@FutureOrPresent`).

Regla extra que vive en `CocheraService`

Si un administrador **deshabilita** una cochera que tenía reservas CONFIRMADAS, el sistema las pasa automáticamente a CANCELADA. Y si intenta **borrar** una cochera que tiene reservas, se lo impide con un error 400: primero hay que resolver esas reservas.

La arquitectura en una servilleta

AparcAR no es un programa: son **tres programas separados** que se hablan por la red. Esto se llama arquitectura cliente-servidor de tres capas, y es como esta hecho practicamente todo lo que usas en internet.

1. NAVEGADOR - lo que ve el usuario (FRONTEND)

Corre en la computadora del usuario, en Chrome o Firefox. Es la carpeta `aparcar-front/`. Esta hecho con Next.js y React. Dibuja botones y formularios. **No sabe nada de la base de datos**. Vive en el puerto 3000.



2. API - el cerebro (BACKEND)

Corre en un servidor. Es la carpeta `aparcar-api-back/`. Esta hecho con Java y Spring Boot. Recibe pedidos del navegador, valida las reglas de negocio, decide quien puede hacer que, y habla con la base de datos. Vive en el puerto 8080.



3. POSTGRESQL - la memoria (BASE DE DATOS)

Guarda los datos en disco para que sobrevivan al apagado. Es un programa aparte, que en este proyecto corre dentro de Docker. Vive en el puerto 5433 visto desde tu maquina.

Los tres procesos se comunican por HTTP y SQL. Cada uno puede reiniciarse sin tocar a los otros.

2.1 La analogia del restaurante

Si nunca viste esta separacion, pensala como un restaurante:

En el restaurante	En AparcAR	Por que
El mozo y la carta	El frontend (Next.js)	Es la cara visible. Toma tu pedido, te lo presenta lindo. No cocina nada.
La cocina	El backend (Spring Boot)	Ahi pasa el trabajo real. Recibe la comanda, verifica que haya ingredientes, aplica las reglas ("no servimos pizza despues de las 12").
La heladera y la despensa	PostgreSQL	Guarda las cosas. No decide nada, solo almacena y entrega cuando le piden.
La comanda escrita	El request HTTP (JSON)	El formato estandar en que el mozo le pasa el pedido a la cocina.
El carnet de empleado	El token JWT	Lo que prueba que quien esta entrando a la cocina trabaja ahi.

Por que separarlo asi y no hacer un solo programa

Porque cada parte se puede cambiar sin romper las otras. Si maniana quieren una app de celular, no hay que reescribir nada del backend: la app le pega a los mismos endpoints que le pega el navegador.

Porque el navegador no es de fiar. Cualquiera puede abrir las herramientas de desarrollador y cambiar el JavaScript. Por eso toda validacion importante se repite en el backend, donde el usuario no puede tocarla. En AparcAR vas a ver validaciones duplicadas (Zod en el front, Bean Validation en el back) y eso **no es un error**: la del front es para dar feedback rapido, la del back es la que realmente protege.

2.2 Que pasa cuando hacés click en "Confirmar reserva"

El recorrido completo, en once pasos. Vuelve a esta pagina cada vez que te pierdas:

1. Hacés click. React ejecuta una funcion de JavaScript en tu navegador.
2. **Zod** revisa el formulario. Si falta la fecha, te marca el error en rojo y **no sale ningun pedido a la red**.
3. Si esta todo bien, **Axios** arma un pedido HTTP: POST `http://localhost:8080/api/v1/reservas` con un cuerpo en formato JSON.
4. El **interceptor** de Axios le pega automaticamente el header `Authorization: Bearer eyJ...` con tu token guardado en una cookie.
5. El pedido viaja por la red al backend, puerto 8080.
6. En el backend lo recibe primero la **cadena de filtros de seguridad**: se valida el token, se saca de ahi tu email y tus roles, y se chequea que tengas permiso para ese endpoint.
7. Spring busca que metodo Java corresponde a esa URL. Encuentra `ReservaController.crear()`.
8. Antes de entrar al metodo, **Bean Validation** revisa el JSON: que no falte la fecha, que no sea pasada. Si falla, 400 y no se ejecuta nada mas.
9. El controller llama a `ReservaService.crear()`, que aplica las cinco reglas de negocio del capitulo anterior.
10. El service le pide a **ReservaRepository** que guarde. Hibernate traduce eso a un `INSERT INTO reservas ...` y PostgreSQL lo escribe en disco.
11. Vuelve todo para atras: el service arma un DTO de respuesta, el controller responde **201 Created**, Axios recibe el JSON, React actualiza la pantalla y **Sonner** muestra el cartelito verde de exito.

Diccionario de tecnologias

Este es el capitulo mas largo y el mas util. Cada entrada responde tres preguntas: **que es** (en criollo), **por que esta en este proyecto**, y **donde lo ves** (archivo concreto). Si en algun momento leyendo codigo aparece una palabra que no conoces, buscala aca.

Concepto previo: que es una "dependencia" o "libreria"

Nadie escribe todo desde cero. Cuando necesitas mandar un mail, no programas el protocolo SMTP: usas codigo que otra gente ya escribio y publico. A ese codigo ajeno se le dice **dependencia** o **libreria**.

Cada proyecto lista sus dependencias en un archivo. En Java ese archivo es `pom.xml`. En JavaScript es `package.json`. Cuando corres el build, una herramienta lee esa lista y baja todo de internet automaticamente. Por eso vas a ver, la primera vez, un monton de lineas de "Downloading...".

3.1 La base del backend

Java 21

Que es. Un lenguaje de programacion compilado, con tipado estatico (tenes que declarar de que tipo es cada cosa, y el compilador te avisa si te equivocas **antes** de ejecutar). Es de los lenguajes mas usados del mundo en sistemas empresariales.

Por que esta en AparcAR. Es el lenguaje en el que esta escrito todo el backend. La version 21 es una LTS (soporte de largo plazo). El proyecto ademas activa *virtual threads*, una capacidad nueva de Java que permite atender muchisimos pedidos simultaneos sin gastar un hilo del sistema operativo por cada uno.

Donde lo ves. Todos los archivos `.java`. La version se declara en `pom.xml`: `<java.version>21</java.version>`. Los virtual threads se activan en `application.yml` con `spring.threads.virtual.enabled: true`.

Spring Boot 4.0.3

Que es. Un **framework**. Un framework es una libreria tan grande que invierte la relacion: en vez de que vos llames a la libreria, la libreria te llama a vos. Spring Boot arranca un servidor web, escanea tus clases, las conecta entre si, lee la configuracion y te deja una aplicacion andando con muy poco codigo propio.

Por que esta en AparcAR. Es el esqueleto de todo el backend. Sin Spring Boot habria que escribir a mano el servidor HTTP, el ruteo de URLs, la conexion a la base, la seguridad y la serializacion a JSON: miles de lineas.

Donde lo ves. `AparcarApiApplication.java` es el punto de entrada: tiene un `main()` de tres lineas que levanta todo.

El concepto mas importante de Spring: inyeccion de dependencias

Mira esta linea, que aparece en casi todos los archivos del backend:

```
private final IReservaService reservaService;
```

El controller **necesita** un service, pero **nunca lo crea** (no hay ningun `new ReservaService(...)`). Al arrancar, Spring recorre todas las clases marcadas con `@Service`, `@Repository`, `@Component` o `@Configuration`, crea una sola instancia de cada una (eso se llama un **bean**) y se las va entregando a quien las pida por el constructor.

Ventaja practica: en los tests podes darle al controller un service falso (un *mock*) sin tocar una linea del codigo de produccion. Eso es exactamente lo que hacen los tests de service/ en este repo.

Maven y el archivo pom.xml

Que es. La herramienta que gestiona el proyecto Java: baja las dependencias, compila el código, corre los tests y empaqueta todo en un archivo `.jar` (un zip ejecutable con tu programa adentro). El `pom.xml` es su archivo de configuración: ahí está la lista de todo lo que el proyecto usa.

Por que esta en AparcAR. Es lo que convierte 130 archivos `.java` sueltos en un programa ejecutable. Comandos típicos: `mvn test`, `mvn clean install`.

Donde lo ves. `aparcAR-api-back/pom.xml`. Ahí puedes leer, en orden, todas las tecnologías que siguen en este capítulo.

Anotaciones (@Algo)

Que es. Las palabras con arroba que vas a ver por todos lados: `@RestController`, `@Entity`, `@Valid`. No son comentarios: son **metadatos que el programa lee de sí mismo en tiempo de ejecución**. Spring las mira y actúa en consecuencia.

Por que esta en AparcAR. Es el mecanismo con el que Spring sabe que hacer con cada clase sin que vos le tengas que explicar nada. La anotación es la explicación.

Donde lo ves. En absolutamente todos los archivos `.java` del backend. Cada vez que veas una arroba, piensa: "aca hay una instrucción para el framework".

3.2 Persistencia: como se guardan los datos

PostgreSQL 16

Que es. Un **sistema gestor de base de datos relacional**. Guarda información en tablas con filas y columnas, como un Excel gigante, pero con reglas estrictas, transacciones (o se guarda todo o no se guarda nada) y un lenguaje propio de consulta: SQL.

Por que esta en AparcAR. Es donde vive de verdad la información de AparcAR. Es un programa aparte del backend; en este proyecto corre en su propio contenedor Docker.

Donde lo ves. `dockerfiles/db.Dockerfile` define la imagen. La conexión se configura con la variable `DB_URL` del archivo `.env`.

JPA (Jakarta Persistence API)

Que es. Un **estándar**, no un programa. Define como se supone que un objeto Java se mapea a una fila de una tabla. Es la especificación: dice que debe existir una anotación `@Entity`, que debe existir `@Id`, etc. Pero no la implementa.

Por que esta en AparcAR. Permite que el código no dependa de un motor concreto. Como el proyecto programa contra JPA y no contra Hibernate directamente, los tests pueden correr contra H2 y la producción contra PostgreSQL sin cambiar una línea.

Donde lo ves. Cada `import jakarta.persistence.*` en la carpeta `entity/`.

Hibernate

Que es. La **implementación** concreta de JPA que usa Spring Boot por defecto. Es un ORM (Object-Relational Mapper): traduce automáticamente entre objetos Java y tablas SQL. Vos escribis `reservaRepository.save(reserva)` y Hibernate genera el `INSERT INTO reservas (fecha, visitante_id, ...) VALUES (...)`, lo manda, y te devuelve el objeto con el id que asignó la base.

Por que esta en AparcAR. Te ahorra escribir SQL a mano para el 95% de las operaciones. Ademas maneja las relaciones: cuando pedis una Reserva, sabe ir a buscar su Visitante, su Vehiculo y su Cochera.

Donde lo ves. No lo importas directamente casi nunca, trabaja por debajo. Pero se ve en `Reserva.java`: la anotacion `@ManyToOne` le dice "muchas reservas apuntan a un mismo visitante", y Hibernate crea la columna `visitante_id`.

Hibernate tiene un interruptor peligroso: `ddl-auto`

`ddl-auto` le dice a Hibernate cuanto puede modificar la estructura de la base por su cuenta. Los valores usados en este proyecto:

validate (produccion, en `application.yml`): "mira la base y avisame si no coincide con mis clases, pero **no toques nada**".

update (desarrollo, en `application-dev.yml`): "si falta una tabla o una columna, creala vos". Comodo para desarrollar, peligroso en produccion porque puede modificar una base con datos reales sin que nadie lo revise.

create-drop (tests, en `application-test.yml`): "borra todo y crea de cero al arrancar, y borra al terminar". Perfecto para tests porque garantiza que cada corrida empieza limpia.

Spring Data JPA y los Repository

Que es. Una capa mas arriba de Hibernate. Abri `repository/ReservaRepository.java`: es una **interfaz vacia**, sin una sola linea de implementacion. Y sin embargo funciona. Spring lee el **nombre** de cada metodo, lo interpreta como si fuera una frase, y genera la consulta solo.

Por que esta en AparcAR. Elimina la capa mas repetitiva y aburrida de cualquier sistema. Cero SQL escrito a mano en todo el proyecto.

Donde lo ves. El caso mas claro es este metodo, que es el que impide la sobreocupacion:

```
boolean existsByCocheraIdAndFechaAndEstado(UUID cocheraId,
                                             LocalDate fecha,
                                             ReservaEstado estado);

// Spring lee el nombre y genera, sin que nadie lo escriba:
// SELECT COUNT(*) > 0 FROM reservas
// WHERE cochera_id = ? AND fecha = ? AND estado = ?
```

Liquibase

Que es. Un **sistema de control de versiones para la estructura de la base de datos**. En vez de que alguien entre a la base y escriba a mano "agregale una columna", se escribe un archivo (un *changeset*) que describe el cambio. Liquibase lleva la cuenta, en una tabla propia llamada `databasechangelog`, de que *changesets* ya aplico, y al arrancar corre solamente los nuevos.

Por que esta en AparcAR. Es lo que permite que cinco personas del equipo tengan la misma estructura de base sin mandarse instrucciones por WhatsApp. Cuando alguien clona el repo y levanta el proyecto, las tablas se crean solas, en orden, igual que en la maquina de todos los demas.

Donde lo ves. `src/main/resources/db/changelog/`. El archivo `db.changelog-master.yml` es el indice que lista los otros tres en orden.

Liquibase vs Hibernate ddl-auto: los dos crean tablas, y eso genera un lio real en este proyecto

En AparcAR conviven **los dos mecanismos**, y eso es una deuda tecnica documentada. Las tablas de negocio (visitantes, vehiculos, cocheras, reservas) las crea **Liquibase**. Las tablas de login (app_users, otp_codes) las crea **Hibernate** con ddl-auto: update, porque la migracion 001-initial-schema.yaml quedo vacia.

El problema: **Liquibase corre primero, Hibernate despues**. Por eso la migracion 003, que agrega la columna visitantes.app_user_id, **no puede** declarar una clave foranea hacia app_users: cuando corre, esa tabla todavia no existe. La relacion la valida JPA en el codigo (@OneToOne en Visitante.java), no la base.

Se descubrio escribiendo los tests: en la base Postgres de desarrollo funcionaba (porque la tabla ya existia de arranques anteriores) pero fallaba siempre en H2, que arranca vacia. Es un ejemplo perfecto de por que los tests valen la pena.

H2

Que es. Una base de datos que corre **dentro de la memoria RAM del propio programa Java**, sin instalar nada y sin escribir nada a disco. Al cerrarse el programa, desaparece.

Por que esta en AparcAR. Es la base que usan los tests. Arranca en milisegundos, cada corrida empieza vacia y no ensucia la base real. Por eso podes correr los 149 tests del backend sin tener Docker ni PostgreSQL levantados.

Donde lo ves. application-test.yml: url: jdbc:h2:mem:testdb. El mem significa "en memoria".

3.3 Seguridad

Spring Security

Que es. El modulo de Spring que se encarga de dos cosas distintas que conviene no confundir: **autenticacion** (quien sos: probame que sos vos) y **autorizacion** (que podes hacer: ya se quien sos, pero esto no te corresponde). Funciona interponiendo una **cadena de filtros** delante de toda la aplicacion.

Por que esta en AparcAR. Es lo que hace que un usuario con rol USER reciba un 403 si intenta entrar a la pantalla de administracion de usuarios, aunque escriba la URL a mano.

Donde lo ves. security/securityConfig/DevSecurityConfig.java y su gemelo ProdSecurityConfig.java. Ahi esta la lista completa de que URL exige que.

HTTP Basic Auth

Que es. La forma mas antigua y simple de autenticarse en la web: se manda el header Authorization: Basic <email:password en base64>. Base64 **no es encriptacion**, es solo una forma de escribir texto con caracteres seguros: cualquiera lo puede revertir. Por eso Basic solo es aceptable sobre HTTPS.

Por que esta en AparcAR. Es el metodo que usa AparcAR **unicamente para el login**. Se manda una sola vez, se recibe un token a cambio, y a partir de ahi ya no se manda mas la contraseña.

Donde lo ves. app/login/page.jsx, linea del btoa() (la funcion de JavaScript que codifica en base64).

JWT (JSON Web Token)

Que es. Un **carnet digital firmado**. Es un texto largo con tres partes separadas por puntos: cabecera.contenido.firma. El contenido es JSON en base64 y **se puede leer sin ninguna clave** (pegalo en jwt.io y vas a ver tu email). Lo que no se puede es **modificarlo**: la firma se calcula con una clave secreta que solo tiene el servidor, y si cambias una coma del contenido la firma deja de dar.

Por que esta en AparcAR. Resuelve el problema de "como se yo, en el pedido numero 500, que sos el mismo que se logueo hace una hora" sin guardar sesiones en el servidor. El servidor de AparcAR es **stateless** (sin estado): no recuerda nada entre pedidos, toda la informacion de quien sos viaja en el token.

Donde lo ves. Se genera en `filters/JWTGeneratorFilter.java` y se valida en `filters/JWTValidationFilter.java`. En el front lo guarda `store/authStore.js` en una cookie llamada JWT.

El JWT que emite AparcAR contiene exactamente esto:

```
issuer      : "AparcAR"
subject     : "JWT Token"
email       : "admin@test.com"      <- quien sos
authorities : "USER,ADMIN"         <- que podes hacer
issuedAt    : ahora
expiration  : ahora + 8 horas      <- vence solo
```

Todo eso, firmado con una clave secreta usando HMAC-SHA.

Ojo con este detalle, que confunde a todo el mundo

El claim `authorities` es **un solo string separado por comas** ("USER,ADMIN"), no un array. Por eso tanto `authStore.js` como `serverAuth.js` en el frontend hacen `.split(",")` para convertirlo en lista. Si algun dia ves un rol que no se reconoce, mira ahi primero.

BCrypt

Que es. Un algoritmo de **hashing** de contraseñas. Un hash es una funcion de una sola via: de la contraseña saca un texto ilegible, pero de ese texto **no se puede volver** a la contraseña. Para verificar un login, se hashea lo que el usuario escribio y se compara con lo guardado. BCrypt ademas es deliberadamente **lento**, para que probar millones de contraseñas por fuerza bruta sea inviable.

Por que esta en AparcAR. Es lo que hace que, si alguien roba la base de datos, no se lleve las contraseñas de nadie.

Donde lo ves. `DevSecurityConfig.java` declara el bean `BCryptPasswordEncoder`, y `ProdAuthProvider.java` lo usa con `passwordEncoder.matches(...)`.

Bucket4j + Caffeine (rate limiting)

Que es. **Bucket4j** implementa el algoritmo del "balde con fichas": cada IP tiene un balde con 5 fichas que se recargan cada minuto; cada pedido consume una; si el balde esta vacio, se rechaza con un codigo 429 (Too Many Requests). **Caffeine** es un cache en memoria: guarda el balde de cada IP y olvida los que no se usan hace 10 minutos.

Por que esta en AparcAR. Frena los ataques de fuerza bruta contra el login. Sin esto, un atacante podria probar miles de contraseñas por segundo.

Donde lo ves. `filters/RateLimitFilter.java`. Solo se aplica a tres rutas: `/login`, `/register` y `/forgot-password`.

Caffeine tambien como cache de tokens revocados

Que es. Un JWT, una vez emitido, es valido hasta que vence: el servidor no lo puede "apagar" porque no guarda ninguna lista de tokens. La solucion habitual es una **lista negra**: si un usuario es dado de baja, se anota su email, y el filtro de validacion rechaza cualquier token de ese email aunque la firma este perfecta.

Por que esta en AparcAR. Permite expulsar a alguien del sistema sin esperar 8 horas a que le venza el token.

Donde lo ves. `component/impl/RevokedUserCache.java`, consultado en `JWTValidationFilter` con `revokedCache.isRevoked(email)`.

3.4 Utilidades del backend

Lombok

Que es. Una libreria que **escribe codigo por vos en tiempo de compilacion**. Poniendo `@Getter` y `@Setter` arriba de una clase, aparecen todos los metodos `getNombre()` / `setNombre()` sin escribirlos. `@RequiredArgsConstructor` genera el constructor con todos los campos `final` (esto es lo que permite la inyeccion de dependencias sin escribir el constructor a mano). `@Data` es todo junto. `@Slf4j` crea el objeto `log` para escribir mensajes.

Por que esta en AparcAR. Sin Lombok, `Reserva.java` tendria unas 120 lineas en vez de 50, casi todas mecanicas.

Donde lo ves. Arriba de casi todas las clases del backend. Si alguna vez tu editor te marca en rojo un `getNombre()` que "no existe", es que le falta activar el procesador de anotaciones de Lombok.

Jakarta Bean Validation

Que es. Un sistema de validacion declarativa: en vez de escribir `if (fecha == null) throw ...`, ponés `@NotNull` arriba del campo. Cuando el controller marca el parametro con `@Valid`, Spring corre todas esas validaciones antes de entrar al metodo y, si alguna falla, responde 400 automaticamente.

Por que esta en AparcAR. Centraliza las validaciones de formato en el DTO, donde se leen de un vistazo, en vez de dispersarlas por el codigo.

Donde lo ves. `dto/reserva/ReservaRequestDto.java`. Mira las anotaciones `@NotNull` y `@FutureOrPresent` con sus mensajes en castellano.

DTO (Data Transfer Object) - es un patron, no una libreria

Que es. Una clase cuyo unico proposito es transportar datos entre el backend y el mundo exterior. Hay dos familias: `...RequestDto` (lo que entra) y `...ResponseDto` (lo que sale).

Por que esta en AparcAR. Es una **medida de seguridad y de diseño**. Si el backend devolviera directamente la entidad `AppUser`, estaria mandando la contraseña hashada al navegador. Por eso existe `UserResponseDto`, que tiene los mismos campos **menos** el password. Ademas desacopla: podés reordenar la base sin romperle el contrato a quien consume la API.

Donde lo ves. Toda la carpeta `dto/`.

ModelMapper

Que es. Copia automaticamente los campos de un objeto a otro cuando se llaman igual (por ejemplo, de una entidad a su DTO).

Por que esta en AparcAR. Ahorra el codigo repetitivo de copiar campo por campo. En este proyecto se usa poco: varios services (como `ReservaService`) prefieren hacer la conversion a mano en un metodo `toResponseDto()`, que es mas explicito y mas facil de debuggear.

Donde lo ves. `config/ModelMapperConfig.java`.

Spring Mail + spring-dotenv

Que es. Spring Mail manda emails por SMTP (el protocolo de correo). **spring-dotenv** lee el archivo `.env` y expone sus valores como si fueran variables de entorno del sistema.

Por que esta en AparcAR. El mail se usa para el flujo de recuperar contraseña: se genera un codigo de un solo uso (OTP, One Time Password) y se manda al mail del usuario. El dotenv existe porque las credenciales (usuario y contraseña de la base, del mail) **nunca** se suben a git: van en `.env`, que esta ignorado, y el repo solo trae `.env.example` como plantilla.

Donde lo ves. `component/impl/SpringEmailSender.java` y `entity/auth/OneTimePassword.java`. La tarea programada `component/OTPCleanup.java` borra periodicamente los codigos vencidos.

SpringDoc OpenAPI (Swagger UI)

Que es. Lee todos tus controllers, deduce la documentacion de la API (que endpoints hay, que reciben, que devuelven) y publica una pagina web interactiva donde se pueden probar los endpoints sin escribir codigo.

Por que esta en AparcAR. Es la mejor forma de explorar el backend cuando estas aprendiendo. Ademas es la verificacion recomendada de que el backend arranco bien.

Donde lo ves. Con el backend corriendo, abri <http://localhost:8080/docs>. La ruta se configura en `application.yml` con `springdoc.swagger-ui.path: /docs`.

Spring Actuator

Que es. Expone endpoints de monitoreo sobre la salud interna de la aplicacion. En este proyecto esta recortado a uno solo: `/actuator/health`, que devuelve si la app esta viva.

Por que esta en AparcAR. Docker lo puede usar para saber si el contenedor esta sano. Esta declarado como publico en la configuracion de seguridad justamente para eso.

Donde lo ves. `application.yml`, seccion `management.endpoints.web.exposure.include: health`.

Spring DevTools, WebClient y AsyncConfig

Que es. DevTools reinicia la aplicacion sola cuando detecta un cambio en el codigo (solo en desarrollo). **WebClient** (que viene con WebFlux) es un cliente HTTP para que el backend **le pegue a otras APIs**, al revés de lo habitual. **AsyncConfig** habilita `@Async`: metodos que se ejecutan en segundo plano sin hacer esperar al usuario.

Por que esta en AparcAR. DevTools acelera el desarrollo. El `@Async` se usa para mandar mails: el usuario recibe su respuesta al instante y el mail sale por atras.

Donde lo ves. `config/AsyncConfig.java`, `config/WebClientConfig.java`. En los tests, `config/SynchronousAsyncConfig.java` desactiva el asincronismo para que los tests sean deterministas.

JaCoCo

Que es. Mide la **cobertura de tests**: que porcentaje de las lineas de tu codigo fueron efectivamente ejecutadas al correr la suite. Genera un reporte HTML navegable donde el codigo cubierto se pinta de verde y el no cubierto de rojo.

Por que esta en AparcAR. Permite ver de un vistazo que partes del sistema nadie esta probando. El `pom.xml` lo configura para **excluir** del calculo las carpetas sin logica (`dto`, `entity`, `config`), porque medir cobertura sobre getters infla el numero sin aportar nada.

Donde lo ves. Se corre con `mvn verify` (o `make cover`) y el reporte queda en `target/site/jacoco/index.html`.

3.5 Testing del backend

JUnit 5

Que es. El framework de tests de Java. Un test es simplemente un metodo marcado con `@Test` que, si termina sin lanzar una excepcion, se considera aprobado. Las **aserciones** (`assertEquals`, `assertThrows`) son las que lanzan la excepcion cuando el resultado no es el esperado.

Por que esta en AparcAR. Es la base de los 149 tests del backend.

Donde lo ves. Todos los archivos de `src/test/java/`. La anotacion `@DisplayName` es la que permite los nombres largos en castellano que ves en `TESTS.md`.

Mockito

Que es. Fabrica objetos falsos (*mocks*). Le decis "cuando alguien llame a `visitanteRepository.findById(x)`, devuelve este visitante inventado" y el mock obedece, sin tocar ninguna base de datos.

Por que esta en AparcAR. Permite testear un service **aislado**: si el test falla, sabes con certeza que el problema esta en la logica del service y no en la base, la red o el mapeo. Es la diferencia entre `ReservaServiceTests` (con mocks, rapido, aislado) y `ReservaControllerTests` (contra base real, lento, realista). El proyecto tiene los dos **a proposito**.

Donde lo ves. `config/UnitTests.java` es la configuracion reutilizable de Mockito.

MockMvc

Que es. Un **servidor web simulado**. Te deja escribir `mockMvc.perform(post("/api/v1/reservas")...)` y ejecutar todo el circuito de Spring (ruteo, validacion, controller, service, base) **sin abrir un puerto de red de verdad**. Por eso es rapidisimo comparado con levantar un servidor.

Por que esta en AparcAR. Es lo que usan casi todos los tests de integracion del proyecto: `CocheraControllerTests`, `ReservaControllerTests`, `VisitanteControllerTests`, etc.

Donde lo ves. `config/IntegrationTests.java` es la clase base que todos heredan.

La trampa de MockMvc que este proyecto descubrio a los golpes (leer dos veces)

Los tres filtros de seguridad de AparcAR (`RateLimitFilter`, `JWTValidationFilter`, `JWTGeneratorFilter`) deciden si aplicarse mirando `request.getServletPath()` dentro de su metodo `shouldNotFilter()`.

En el despacho simulado de MockMvc, ese valor **queda vacio** y no coincide con el de un servidor real. Resultado: **esos tres filtros nunca se ejecutan bajo MockMvc**. Y no falla nada, no hay error, no hay warning. Simplemente no hacen nada, en silencio.

Por eso `LoginFlowTests` es el unico test del proyecto que levanta un **servidor embebido real** (`@SpringBootTest(webEnvironment = RANDOM_PORT)` mas `RestTemplate`): es la unica manera de que esos filtros corran de verdad durante un test.

La leccion general: un test que pasa no siempre prueba lo que vos crees que prueba. Esto no es un bug de produccion (contra la app real en Docker el JWT y el rate limiting funcionan), es una limitacion del entorno de test.

3.6 El frontend

Node.js y npm

Que es. **Node.js** es un programa que ejecuta JavaScript fuera del navegador. **npm** es su gestor de paquetes: el equivalente de Maven en el mundo JavaScript. Lee `package.json`, baja las dependencias a la carpeta

node_modules/ y corre los scripts.

Por que esta en AparcAR. Next.js corre sobre Node. Sin Node no hay frontend.

Donde lo ves. aparc-ar-front/package.json. Los comandos disponibles son npm run dev, npm run build, npm start, npm test.

React 19

Que es. Una libreria para construir interfaces. Su idea central: la pantalla es una **funcion del estado**. Vos no decis "agrega una fila a la tabla"; decis "el estado ahora tiene 4 reservas" y React se encarga de redibujar lo que haga falta. Las piezas se llaman **componentes**: funciones de JavaScript que devuelven algo parecido a HTML (se llama JSX).

Por que esta en AparcAR. Es lo que dibuja todas las pantallas de AparcAR.

Donde lo ves. Todos los archivos .jsx. Por ejemplo app/dashboard-admin/EstadoCocherasGrid.jsx.

Next.js 16 (App Router)

Que es. Un framework construido encima de React que agrega dos cosas grandes. **Ruteo por carpetas**: la URL sale de la estructura de directorios, no de un archivo de configuracion. Y **renderizado en el servidor**: hay componentes que se ejecutan en el servidor Node antes de mandarle nada al navegador.

Por que esta en AparcAR. El ruteo por carpetas es la razon de que el frontend este organizado asi. Y el renderizado en servidor es lo que permite la proteccion de rutas mas solida: la pagina de administracion de usuarios verifica el rol **en el servidor**, antes de mandar una sola letra de HTML.

Donde lo ves. La regla es simple: una carpeta dentro de app/ con un archivo page.jsx adentro se convierte en una URL.

```
app/login/page.jsx           -> /login
app/dashboard-admin/page.jsx -> /dashboard-admin
app/dashboard-admin/cocheras/page.jsx -> /dashboard-admin/cocheras
app/dashboard-admin/usuarios/page.jsx -> /dashboard-admin/usuarios
app/dashboard-user/page.jsx  -> /dashboard-user
```

Los archivos que NO se llaman page.jsx (EstadoCocherasGrid.jsx, UserManagement.jsx, ...) no son URLs: son componentes que las paginas importan y usan adentro.

"use client": la linea de una sola palabra que divide el frontend en dos

Si un archivo .jsx empieza con "use client", se ejecuta **en el navegador** y puede usar interactividad: useState, manejadores de click, formularios. Si no la tiene, es un **Server Component**: se ejecuta en el servidor Node, puede leer cookies y hacer redirecciones, pero no puede tener un boton con onClick.

En AparcAR el patron es siempre el mismo: page.jsx es de servidor y solo hace la verificacion de rol; el componente pesado (UserManagement.jsx, CocherasManagement.jsx) es de cliente y hace toda la interaccion. Entender esto explica por que hay dos archivos donde parece que alcanzaria con uno.

Tailwind CSS 4

Que es. Una forma distinta de escribir estilos. En vez de crear una clase .boton-azul en un archivo CSS aparte, ponés clases chiquitas y descriptivas directo en el HTML: class="flex w-full rounded-xl bg-[#0cb7f2] px-3 py-3" significa "caja flexible, ancho completo, bordes redondeados, fondo cyan, padding horizontal 3 y vertical 3".

Por que esta en AparcAR. Al principio parece ilegible, pero evita el problema clasico de tener un archivo CSS de 3000 lineas que nadie se anima a tocar porque no sabe que pantalla va a romper.

Donde lo ves. Los atributos `className` de todos los `.jsx`. Los colores de marca del proyecto son `#002147` (azul oxford) y `#0cb7f2` (cyan).

Axios

Que es. Una libreria para hacer pedidos HTTP desde JavaScript. Su funcion estrella son los **interceptores**: codigo que se ejecuta automaticamente antes de cada pedido que sale y despues de cada respuesta que llega.

Por que esta en AparcAR. AparcAR tiene exactamente dos interceptores, y valen oro: uno **le pega el token JWT a todo pedido saliente** (asi ningun componente tiene que acordarse de hacerlo), y otro **detecta las respuestas 401** (token vencido) para borrar la cookie y mandar al usuario al login, desde cualquier pantalla.

Donde lo ves. `app/api.jsx`. Son 40 lineas y son de lo mas importante del frontend: leelas enteras.

Zustand

Que es. Un gestor de **estado global**. El estado normal de React vive dentro de un componente; si otro componente lo necesita, hay que pasarlo de padre a hijo, a veces por cinco niveles. Zustand crea una cajita a la que cualquier componente accede directo.

Por que esta en AparcAR. Guarda la sesion: si estas autenticado, quien sos, que roles tenes. Lo necesitan la barra de navegacion, los dashboards y las rutas protegidas por igual.

Donde lo ves. `store/authStore.js`. Expone tres funciones: `setAuth(token)` al loguearse, `logout()`, y `checkAuth()` que restaura la sesion desde la cookie al recargar la pagina (y la cierra sola si el token ya vencio).

React Hook Form + Zod + @hookform/resolvers

Que es. **React Hook Form** maneja formularios sin redibujar toda la pantalla en cada tecla. **Zod** define un esquema de validacion como un objeto: `z.string().email("Ingresa un correo valido")`. **@hookform/resolvers** es el pegamento que conecta los dos.

Por que esta en AparcAR. Es lo que hace que veas los errores en rojo debajo de cada campo antes de mandar nada. Recordá: **esta validacion es solo comodidad**, la validacion real vive en el backend con Bean Validation.

Donde lo ves. `app/login/page.jsx` tiene el ejemplo mas claro: el `loginSchema` de cinco lineas al principio del archivo.

Sonner y jwt-decode

Que es. **Sonner** muestra las notificaciones flotantes (los "toasts"): el cartelito verde de exito o rojo de error que aparece arriba y se va solo. **jwt-decode** lee el contenido de un JWT sin verificar la firma.

Por que esta en AparcAR. Sonner da el feedback de cada accion. `jwt-decode` es lo que permite que el frontend sepa tu rol para decidir a que dashboard mandarte.

Donde lo ves. El componente `<Toaster />` se monta una sola vez en `app/layout.js` y despues cualquier archivo puede hacer `toast.success(...)`.

Por que el frontend puede leer el JWT sin la clave secreta (y por que no importa)

`jwt-decode` **no verifica nada**: solo decodifica el base64 del medio. Un usuario malicioso podria editar su token para poner `"authorities": "ADMIN"` y el frontend le mostraria el menu de administrador.

Y no pasa nada. Porque cuando ese menu intente pedir datos, el **backend** va a verificar la **firma** con la clave secreta, va a ver que no da, y va a responder 401. La pantalla se veria pero estaria vacia.

Esta es la regla de oro de la seguridad web: la validacion del frontend es para la experiencia de usuario; la del backend es la que protege. Nunca al reves.

3.7 Testing del frontend

Vitest

Que es. El corredor de tests del frontend: el equivalente de JUnit. Encuentra los archivos `.test.jsx`, los ejecuta y reporta.

Por que esta en AparcAR. Corre los 77 tests del frontend con `npm test`.

Donde lo ves. `vitest.config.mjs` y `test/setup.js` (que limpia el DOM y las cookies despues de cada test, para que no se contaminen entre si).

React Testing Library + jsdom + @testing-library/user-event

Que es. `jsdom` simula un navegador entero dentro de Node: hay `document`, hay `window`, hay cookies, pero no hay ventana. **React Testing Library** permite buscar cosas en esa pantalla falsa como lo haria un usuario ("busca el boton que dice Ingresar"), no como lo haria un programador ("busca el div con id x"). **user-event** simula tipeo y clicks realistas.

Por que esta en AparcAR. Permiten testear que **la pantalla se comporta bien** sin abrir un navegador de verdad.

Donde lo ves. Por ejemplo `test/login/page.test.jsx` verifica que al escribir mail y contraseña y hacer click, el pedido sale con Basic Auth y no con un body JSON.

axios-mock-adapter

Que es. Intercepta las llamadas de Axios y devuelve respuestas inventadas, sin red.

Por que esta en AparcAR. Permite testear "que hace la pantalla si el backend responde 401" o "si responde que la patente esta duplicada" sin necesitar un backend prendido ni provocar esos errores de verdad.

Donde lo ves. `test/api.test.jsx`, que prueba los dos interceptores de `app/api.jsx`.

ESLint, pre-commit, Make y Task

Que es. **ESLint** revisa el JavaScript en busca de errores y malas practicas sin ejecutarlo. **pre-commit** instala ganchos en git que corren validaciones automaticamente antes de cada commit (valida YAML, **detecta secretos** como contraseñas pegadas por error, limpia espacios). **Make** y **Task** son atajos: `make test` en vez de acordarse del comando largo.

Por que esta en AparcAR. Son la red de seguridad del equipo: hacen que ciertos errores no lleguen nunca al repo.

Donde lo ves. `eslint.config.mjs`, `.pre-commit-config.yaml` (hay uno en cada mitad), `Makefile` y `Taskfile.yaml`.

El arbol de carpetas, comentado

Esta es la parte que hay que tener a mano. La regla general que ordena todo el proyecto es: **cada carpeta agrupa archivos por su responsabilidad, no por el tema del que hablan**. Es decir, todos los controllers juntos, todos los services juntos; no "todo lo de reservas junto".

4.1 Vista general

```
aparcAR/  
|  
|-- README.md ..... Guia de instalacion paso a paso (empeza por aca)  
|-- ARCHIVOS.md ..... Que hace CADA archivo del repo, uno por uno  
|-- TESTS.md ..... Que prueba CADA test, caso por caso (226 en total)  
|-- .gitignore ..... Lista de archivos que git debe ignorar  
|  
|-- aparcAR-api-back/ ..... EL BACKEND. Java + Spring Boot. Puerto 8080.  
|  
+-- aparcAR-front/ ..... EL FRONTEND. Next.js + React. Puerto 3000.
```

Que son las carpetas que no aparecen en este arbol

Si mirás el repo en tu maquina vas a ver tres carpetas que aca no figuran, porque **no son codigo** sino resultados generados automaticamente:

node_modules/ (frontend): las dependencias que bajo npm. Suelen ser cientos de megas y miles de carpetas. Se regenera con `npm install`.

target/ (backend): donde Maven deja las clases compiladas y el `.jar` final. Se regenera con `mvn package`.

.next/ (frontend): el resultado de compilar Next.js. Se regenera con `npm run build`.

Las tres estan en el `.gitignore`. **Nunca se suben a git**, porque cualquiera las puede reconstruir desde el codigo fuente. Si alguna vez las borras por accidente, no pasa absolutamente nada.

4.2 El backend, por fuera

```
aparcar-api-back/
|
|-- pom.xml ..... Lista de dependencias y config de Maven
|-- docker-compose.yml ..... Define los 2 servicios: db (Postgres) y server (la API)
|-- Makefile ..... Atajos: make run, make test, make cover, make migrate
|-- Taskfile.yml ..... Lo mismo que el Makefile, con la herramienta Task
|-- .env ..... Credenciales REALES. NO se sube a git.
|-- .env.example ..... Plantilla del anterior, con valores de ejemplo que funcionan
|-- .pre-commit-config.yaml ... Validaciones automaticas antes de cada commit
|-- README.md ..... Documentacion especifica del backend
|
|-- dockerfiles/
|   |-- db.Dockerfile ..... Receta de la imagen de PostgreSQL
|   |-- db-start.sh ..... Script que corre al iniciar el contenedor de la base
|   +-- server.Dockerfile ..... Receta de la imagen del backend (compila y empaqueta)
|
|-- scripts/
|   +-- create-migration.go ... Generador de migraciones Liquibase (make migrate)
|
+-- src/
    |-- main/ ..... EL CODIGO QUE CORRE EN PRODUCCION
    +-- test/ ..... LOS TESTS (no se empaquetan en el .jar final)
```

4.3 El backend, por dentro: el codigo Java

Esta es la carpeta que hay que entender. Leela de arriba a abajo al menos una vez. Fijate que la organizacion sigue exactamente el camino que recorre un pedido: **controller** (entra), **service** (se decide), **repository** (se guarda). Va partido en tres por su largo, pero es una sola carpeta.

a) El camino del pedido: controller, service, repository

```

src/main/java/com/aparcAR/api/
|
|-- AparcarApiApplication.java .... EL PUNTO DE ENTRADA. El main(). 3 lineas.
|
|-- controller/ ..... LA PUERTA DE ENTRADA HTTP
|   |
|   | Define las URLs. NO tiene logica de negocio:
|   | recibe, delega al service, y devuelve.
|   |
|   |-- AuthController.java ..... /register /login /forgot-password /reset-password
|   |-- CocheraController.java .... ABM de cocheras + /disponibles
|   |-- ReservaController.java .... Alta y consulta de reservas
|   |-- UserController.java ..... Gestion ADMIN de cuentas de usuario
|   |-- VehiculoController.java ... Alta y consulta de vehiculos
|   +-- VisitanteController.java .. Visitantes + los endpoints /me (autoservicio)
|
|-- service/ ..... LA LOGICA DE NEGOCIO. EL CORAZON DEL SISTEMA.
|   |
|   | Aca viven TODAS las reglas. Si querias saber
|   | "por que el sistema no me deja hacer X",
|   | la respuesta esta en esta carpeta.
|   |
|   |-- IAuthService.java ..... Interfaz: el CONTRATO (que se puede hacer)
|   |-- ICocheraService.java ..... Las interfaces empiezan con I por convencion.
|   |-- IReservaService.java ..... Existen para poder cambiar la implementacion
|   |-- IUserService.java ..... sin tocar a quien la usa (y para testear).
|   |-- IVehiculoService.java
|   +-- IVisitanteService.java
|
|   +-- impl/ ..... Interfaz: la IMPLEMENTACION (como se hace)
|       |-- AuthService.java ..... Registro, login, OTP de recuperacion
|       |-- CocheraService.java ... ABM + disponibilidad + cancelar reservas al deshabilitar
|       |-- ReservaService.java ... LAS 5 REGLAS DE ORO DE LAS RESERVAS
|       |-- UserService.java ..... ABM de cuentas + borrado seguro
|       |-- VehiculoService.java .. Patente unica, normalizacion a mayusculas
|       +-- VisitanteService.java . Documento unico + los metodos "propio" (/me)
|
|-- repository/ ..... EL ACCESO A DATOS
|   |
|   | Interfaces VACIAS. Spring Data JPA les genera
|   | la implementacion leyendo el nombre del metodo.
|   |
|   |-- AppUserRepository.java
|   |-- CocheraRepository.java
|   |-- OneTimePasswordRepository.java
|   |-- ReservaRepository.java .... existsByCocheraIdAndFechaAndEstado(...)
|   |-- VehiculoRepository.java
|   +-- VisitanteRepository.java

```

b) Los datos: entity y dto

```

src/main/java/com/aparcAR/api/    (continuacion)
|
|-- entity/ ..... LAS TABLAS, COMO CLASES JAVA
| |
| | |-- auth/
| | |
| | | |-- AppUser.java ..... Tabla app_users: las cuentas de login
| | | |-- AppAuthority.java ..... enum: USER | ADMIN
| | | |-- OneTimePassword.java .. Tabla otp_codes: codigos de recuperacion
| | | +-- InactiveUsersDto.java . Wrapper con los emails inactivos
| |
| | +-- reserva/
| | |
| | | |-- Visitante.java ..... Tabla visitantes (+ el link opcional a AppUser)
| | | |-- Vehiculo.java ..... Tabla vehiculos
| | | |-- VehiculoTipo.java ..... enum: AUTO | MOTO | CARGA
| | | |-- Cochera.java ..... Tabla cocheras
| | | |-- CocheraTipo.java ..... enum: AUTO | MOTO | ACCESIBLE | CARGA
| | | |-- CocheraEstado.java .... enum: habilitada / deshabilitada
| | | |-- Reserva.java ..... Tabla reservas
| | | +-- ReservaEstado.java .... enum: CONFIRMADA | CANCELADA
| |
|-- dto/ ..... LO QUE ENTRA Y SALE POR LA API
| |
| | |-- ErrorResponseDto.java ..... NUNCA se exponen las entidades directamente.
| | |                               Formato unico de error: code, message, details
| | |-- auth/ ..... RegistrationDto, UserResponseDto (SIN password),
| | |                               UpdateUserDto, ResetPasswordDto, UserEmailDto
| | |-- email/ ..... PlainEmailData
| | +-- reserva/ ..... Un par Request/Response por cada concepto:
| | |                               Visitante, Vehiculo, Cochera, Reserva
|

```

c) Lo transversal: security, filters, component, config, exception, events

```

src/main/java/com/aparcAR/api/    (continuacion)
|
|-- security/ ..... QUIEN SOS Y QUE PODES HACER
| |-- AppUserDetailsService.java  Puente: busca un AppUser por email y lo convierte
| |                               al formato que entiende Spring Security
| |-- CustomBasicAuthenticationEntryPoint.java .. La respuesta del 401
| |-- authenticationProvider/
| | |-- DevAuthenticationProvider.java .. DEV/TEST: NO valida la contraseña (!)
| | |-- ProdAuthenticationProvider.java . PROD: valida con BCrypt
| +-- securityConfig/
| | |-- DevSecurityConfig.java .. LA TABLA DE PERMISOS de desarrollo
| | |-- ProdSecurityConfig.java . La equivalente de produccion/test
|
|-- filters/ ..... LO QUE SE EJECUTA ANTES DE TODO PEDIDO
| |-- RateLimitFilter.java ..... Max 5 intentos por IP por minuto en login
| |-- JWTValidationFilter.java .. Lee el token del header y te "loguea"
| |-- JWTGeneratorFilter.java ... Emite el token cuando el login sale bien
| |-- ApiVersionFilter.java ..... Agrega el header X-API-Version
| +-- StripPortFromXffFilter.java  Normaliza la IP detras de un proxy
|
|-- component/ ..... PIEZAS REUTILIZABLES
| |-- IEmailSender.java ..... Contrato de envio de mails
| |-- IRevokedUserCache.java .... Contrato de la lista negra de tokens
| |-- OTPCleanup.java ..... Tarea programada: borra OTPs vencidos
| +-- impl/ ..... RevokedUserCache (Caffeine), SpringEmailSender
|
|-- config/ ..... CONFIGURACION DE SPRING
| |-- ApplicationConstants.java . Constantes: nombres de perfiles, config JWT
| |-- AsyncConfig.java ..... Habilita @Async (tareas en segundo plano)
| |-- SchedulingConfig.java ..... Habilita @Scheduled (tareas programadas)
| |-- ModelMapperConfig.java .... Bean de ModelMapper
| |-- WebClientConfig.java ..... Bean de WebClient
| |-- WebConfig.java ..... Headers X-Forwarded-* (proxies)
| +-- middleware/
| | |-- DevExceptionHandler.java .. Errores con detalle completo (desarrollo)
| | |-- ProdExceptionHandler.java . Errores genericos (produccion, no filtra info)
|
|-- exception/ ..... LOS ERRORES PROPIOS DEL DOMINIO
| |-- NotFoundException.java .... -> se traduce a HTTP 404
| |-- ValidationException.java .. -> se traduce a HTTP 400
| |-- OTPException.java ..... Errores de codigo de recuperacion
| +-- OTPExceptionReason.java ... Los motivos posibles
|
+-- events/ ..... LISTENERS DE LOGS DE SEGURIDAD
| |-- AuthenticationEventsListener.java .. Registra logins exitosos y fallidos
| +-- AuthorizationEventsListener.java ... Registra accesos rechazados

```

4.4 El backend: configuracion y migraciones

```

src/main/resources/
|
|-- application.yml ..... Config BASE, comun a todos los entornos.
| |                               Datasource, mail, Swagger en /docs, ddl-auto: validate
|-- application-dev.yml ..... Solo desarrollo: show-sql (imprime el SQL),
| |                               ddl-auto: update
|-- application-prod.yml ..... Solo produccion
|-- application-test.yml ..... Solo tests: base H2 en memoria, ddl-auto: create-drop
|-- banner.txt ..... El dibujo ASCII que aparece al arrancar
|
+-- db/changelog/ ..... LAS MIGRACIONES DE LIQUIBASE
| |-- db.changelog-master.yaml ..... El INDICE. Lista los otros en orden.
| |-- 001-initial-schema.yaml ..... VACIO (deuda tecnica: las tablas de auth
| |                               las termina creando Hibernate)
| |-- 002-visitantes-vehiculos-cocheras-reservas.yaml .. Las 4 tablas de negocio
| +-- 003-visitante-app-user.yaml .. Agrega visitantes.app_user_id
| |                               (sin FK fisica, a proposito)

```

Como elige Spring cual application-*.yml usar: los perfiles

Spring siempre lee `application.yml`. Además, lee **uno** de los otros tres según el **perfil activo**, y lo que diga ese archivo **pisa** lo que decía el base.

El perfil se elige con la variable de entorno `SPRING_PROFILES_ACTIVE`, que en este proyecto se define en el archivo `.env` (por defecto: `dev`).

Los perfiles no solo cambian configuración: también cambian **que clases existen**. La anotación `@Profile("dev")` arriba de una clase significa "esta clase solo se crea si el perfil activo es `dev`". Por eso `DevAuthenticationProvider` y `ProdAuthenticationProvider` pueden coexistir en el repo sin pisarse: nunca están vivos los dos a la vez.

4.5 El frontend completo

a) Configuración y páginas

```
aparcar-front/
|
|-- package.json ..... Dependencias y scripts (dev, build, start, test)
|-- next.config.mjs ..... Config de Next.js (output: standalone para Docker)
|-- vitest.config.mjs ..... Config de los tests: entorno jsdom, alias @/
|-- postcss.config.mjs ..... Config de Tailwind CSS 4
|-- eslint.config.mjs ..... Reglas del linter
|-- jsconfig.json ..... Define el alias @/ para importar sin ../../..
|-- Dockerfile ..... Receta de la imagen del frontend
|-- docker-compose.yml ..... Levanta el front en el puerto 3000
|-- .env / .env.example ..... NEXT_PUBLIC_API_BASE_URL: donde esta el backend
|-- kickstart.md ..... Guia rapida de instalacion
|
|-- app/ ..... LAS PAGINAS. La estructura de carpetas ES el ruteo.
|   |-- layout.js ..... Envuelve TODAS las paginas. Monta el <Toaster/>.
|   |-- page.js ..... La raiz: /
|   |-- globals.css ..... Estilos globales y la importacion de Tailwind
|   |-- favicon.ico
|   |-- api.js ..... **CLAVE**: la instancia de Axios con los dos
|   |                       interceptores (pega el JWT / maneja el 401)
|   |
|   |-- login/page.jsx ..... /login - Basic Auth y redireccion por rol
|   |-- recover-password/ ..... /recover-password - pedir el codigo OTP
|   |-- reset-password/ ..... /reset-password - cambiar la clave con el OTP
|   |-- unauthorized/ ..... /unauthorized - "no tenes permiso"
|   |
|   |-- dashboard-admin/ ..... SECCION DE ROL ADMIN
|   |   |-- page.jsx ..... Server Component: requireAuth(["ADMIN"])
|   |   |-- EstadoCocherasGrid.jsx .. Cuadrícula visual de ocupacion del día
|   |   |-- VisitantesContent.jsx ... Alta de visitante + vehículo (2 POST seguidos)
|   |   |-- cocheras/
|   |       |-- page.jsx ..... Valida ADMIN en el servidor
|   |       +-- CocherasManagement.jsx .. ABM completo de cocheras + filtros
|   |   +-- usuarios/
|   |       |-- page.jsx ..... Valida ADMIN en el servidor
|   |       +-- UserManagement.jsx .. ABM de cuentas, roles, activar, eliminar
|   |
|   +-- dashboard-user/ ..... SECCION DE ROL USER
|       |-- page.jsx ..... Server Component: requireAuth(["USER"])
|       |-- MiPerfilContent.jsx .. El visitante carga SUS datos y SUS vehiculos
|       +-- ReservasContent.jsx .. Reserva por patente (autocompleta el resto)
```

b) Estado, utilidades y tests

```

aparcAR-front/  (continuacion)
|
|-- store/
|   +-- authStore.js ..... Estado global de sesion (Zustand) + cookie JWT
|
|-- utils/
|   |-- env.js ..... Resuelve las variables NEXT_PUBLIC_* en dev y Docker
|   +-- serverAuth.js ..... requireAuth(roles): la proteccion SERVER-SIDE
|
|-- components/
|   +-- ProtectedRoute.jsx .... Proteccion client-side (el metodo mas viejo)
|
|-- public/
|   +-- Logo.jpeg ..... El logo de la pantalla de login
|
|-- scripts/
|   |-- entrypoint.sh ..... Inyecta las variables NEXT_PUBLIC_* en Docker
|   +-- entrypoint_local.sh ... La variante para desarrollo local
|
+-- test/ ..... LOS 77 TESTS. Espeja la estructura de app/.
    |-- setup.js ..... Limpia el DOM y las cookies entre test y test
    |-- api.test.jsx ..... Los interceptores de Axios
    |-- login/page.test.jsx ... La pantalla de login
    |-- store/authStore.test.js
    |-- utils/env.test.js
    |-- dashboard-admin/ ..... EstadoCocherasGrid, VisitantesContent,
    |                           cocheras/CocherasManagement, usuarios/UserManagement
    +-- dashboard-user/ ..... MiPerfilContent, ReservasContent

```

De donde salen los endpoints del backend

Esta es una de las preguntas mas importantes, porque la respuesta no es obvia: **no existe ningun archivo con una lista de URLs**. No hay un "rutas.txt", no hay una tabla de ruteo. Las URLs **emergen** de las anotaciones que estan escritas arriba de los metodos de cada controller.

5.1 El mecanismo, en tres pasos

1. **Al arrancar**, Spring Boot escanea todas las clases del paquete `com.aparcar.api` buscando anotaciones.
2. Cuando encuentra una clase con `@RestController`, lee su `@RequestMapping("...")` (el prefijo comun) y despues recorre cada metodo buscando `@GetMapping`, `@PostMapping`, `@PutMapping` o `@DeleteMapping`.
3. Con eso arma **en memoria** una tabla: "verbo HTTP + ruta = este metodo Java". Cuando llega un pedido, busca en esa tabla y llama al metodo correspondiente.

Leelo en un ejemplo real: ReservaController.java

```
@RestController                                // 1. Soy un controller REST:
                                              //    lo que devuelva mis metodos se
                                              //    convierte automaticamente a JSON
@RequestMapping("/api/v1/reservas")          // 2. Todas mis rutas empiezan asi
@RequiredArgsConstructor                        // 3. Lombok: genera el constructor
public class ReservaController {

    private final IReservaService reservaService; // Spring me lo inyecta solo

    @PostMapping                                // -> POST /api/v1/reservas
    public ResponseEntity<ReservaResponseDto> crear(
        @Valid                                // valida el DTO antes de entrar
        @RequestBody                          // el JSON del cuerpo -> objeto Java
        ReservaRequestDto dto) {
        return ResponseEntity
            .status(HttpStatus.CREATED)         // codigo 201
            .body(reservaService.crear(dto)); // delega TODO al service
    }

    @GetMapping                                // -> GET /api/v1/reservas
    public ResponseEntity<List<ReservaResponseDto>> listar() {
        return ResponseEntity.ok(reservaService.listar());
    }

    @GetMapping("/{id}")                      // -> GET /api/v1/reservas/{id}
    public ResponseEntity<ReservaResponseDto> obtenerPorId(
        @PathVariable UUID id) {              // toma el {id} de la URL
        return ResponseEntity.ok(reservaService.obtenerPorId(id));
    }
}
```

Fijate lo que este controller NO hace

No valida nada a mano (lo hace `@Valid`). No habla con la base. No decide si la cochera esta libre. No arma el JSON. Tiene **tres lineas de logica** en total, y las tres son "llamale al service". Eso es un controller bien escrito: es un **portero**, no un cerebro. Toda la inteligencia vive en `ReservaService`.

Si algun dia te toca buscar un bug: si es un problema de **ruta o de formato**, mira el controller; si es un problema de **regla de negocio**, el controller no te va a servir de nada, anda derecho al service.

5.2 Los verbos HTTP, y por que importan

La misma URL hace cosas distintas segun el verbo con que la llares. Es la convencion REST y se respeta en todo el proyecto:

Verbo	Significa	Ejemplo en AparcAR	Codigo de exito
GET	Dame informacion. No modifica nada.	GET /api/v1/cocheras	200 OK
POST	Crea algo nuevo.	POST /api/v1/reservas	201 Created
PUT	Modifica algo que ya existe.	PUT /api/v1/cocheras/{id}	200 OK
DELETE	Borra.	DELETE /api/v1/cocheras/{id}	204 No Content

5.3 Tabla completa de endpoints de AparcAR

Esto es todo lo que el backend sabe hacer. La columna "Quien puede" sale de la configuracion de seguridad, que se explica en el capitulo siguiente.

Endpoint	Que hace	Quien puede	Archivo
POST /register	Crea una cuenta nueva (nace INACTIVA y con rol USER)	ADMIN	AuthController
POST /login	Valida usuario y contraseña, devuelve el JWT en el header	Cualquiera con credenciales	AuthController
POST /forgot-password	Manda un codigo OTP al mail	Publico	AuthController
POST /reset-password	Cambia la contraseña usando el OTP	Publico	AuthController
POST /users/activate	Activa una cuenta	ADMIN	UserController
GET /users/inactive	Lista los emails sin activar	ADMIN	UserController
DELETE /users	Elimina una cuenta (no podes borrarte a vos mismo)	ADMIN	UserController
GET /api/v1/usuarios	Lista todas las cuentas (sin password)	ADMIN	UserController
PUT /api/v1/usuarios/{id}	Edita nombre, telefono y roles	ADMIN	UserController
POST /api/v1/cocheras	Crea una cochera (numero unico)	ADMIN	CocheraController
GET /api/v1/cocheras	Lista todas las cocheras	ADMIN	CocheraController
GET /api/v1/cocheras/{id}	Una cochera por id	ADMIN	CocheraController
PUT /api/v1/cocheras/{id}	Edita (si la deshabilita, cancela reservas)	ADMIN	CocheraController
DELETE /api/v1/cocheras/{id}	Borra (falla si tiene reservas)	ADMIN	CocheraController
GET /api/v1/cocheras/disponibles	Cocheras libres para una fecha y tipo de vehiculo	PUBLICO	CocheraController
POST /api/v1/visitantes	Alta de visitante (documento unico)	Autenticado	VisitanteController
GET /api/v1/visitantes	Lista de visitantes	Autenticado	VisitanteController
GET /api/v1/visitantes/{id}	Un visitante por id	Autenticado	VisitanteController
GET /api/v1/visitantes/me	Mi propio perfil de visitante	Autenticado	VisitanteController
POST /api/v1/visitantes/me	Cargo mi propio perfil (1 sola vez)	Autenticado	VisitanteController
POST /api/v1/vehiculos	Alta de vehiculo (patente unica, a mayusculas)	Autenticado	VehiculoController
GET /api/v1/vehiculos	Lista, con filtro opcional ?visitanteld=	Autenticado	VehiculoController
GET /api/v1/vehiculos/{id}	Un vehiculo por id	Autenticado	VehiculoController
POST /api/v1/reservas	Crea la reserva (aplica las 5 reglas)	Autenticado	ReservaController
GET /api/v1/reservas	Lista todas las reservas	Autenticado	ReservaController
GET /api/v1/reservas/{id}	Una reserva por id	Autenticado	ReservaController

Endpoint	Que hace	Quien puede	Archivo
GET /docs	Swagger UI: la API documentada e interactiva	Publico	SpringDoc
GET /actuator/health	Chequeo de salud del servicio	Publico	Actuator

Por que /api/v1/cocheras/disponibles es publico y el resto no

Fijate en el orden de las reglas en DevSecurityConfig.java: la lista de rutas publicas esta **primero**, antes de la regla que exige ADMIN para /api/v1/cocheras/**.

Spring Security evalua las reglas **en orden y se queda con la primera que coincide**. Si estuvieran al reves, /disponibles caeria en la regla de ADMIN y nadie sin ese rol podria consultar la disponibilidad. Hay un comentario en el codigo advirtiendolo. **El orden de las reglas de seguridad no es cosmetico: cambia el comportamiento.**

Como funciona la seguridad, paso a paso

La seguridad de AparcAR tiene dos momentos que conviene separar mentalmente: el **login** (pasa una sola vez) y **todos los demas pedidos** (pasan siempre). Funcionan de manera distinta.

6.1 Momento 1: el login

El usuario escribe mail y contraseña y hace click en Ingresar

Zod valida el formato. Si esta bien, sigue.



El frontend codifica "email:password" en base64

Con la funcion `btoa()` de JavaScript. Manda `POST /login` con el header `Authorization: Basic <...>` y el cuerpo vacio. Archivo: `app/login/page.jsx`.



RateLimitFilter cuenta el intento

Si esa IP ya hizo 5 intentos en el ultimo minuto, corta aca con un 429 y el pedido nunca llega al resto.



Spring Security decodifica el Basic y llama al AuthenticationProvider

En `dev/test` corre `DevAuthenticationProvider`; en `produccion`, `ProdAuthenticationProvider`, que compara la contraseña con `BCrypt`.



AppUserDetailsService busca el usuario por email en la base

Si no existe, se corta con 401. Si existe, devuelve sus roles.



JWTGeneratorFilter arma el token y lo pone en la RESPUESTA

En el header `Authorization: Bearer eyJ...`, con el email, los roles y un vencimiento de 8 horas.



El frontend guarda el token en una cookie llamada JWT

`authStore.setAuth(token)`. Cookie con `max-age` de 7 dias y `SameSite=Lax`.



Redirecciona segun el rol

Si el token trae `ADMIN` va a `/dashboard-admin`; si no, a `/dashboard-user`.

Un detalle que sorprende: en desarrollo, CUALQUIER contraseña funciona

Abri `DevAuthenticationProvider.java`: busca al usuario por email y, si existe, **devuelve la autenticacion sin comparar la contraseña**. No hay ningun `passwordEncoder.matches(...)`.

Es **deliberado**, para no tener que andar recordando claves mientras se desarrolla, y esta **documentado con un test** que se llama textualmente "autentica exitosamente sin importar la contraseña, si el usuario existe". Documentar con un test un comportamiento inseguro-a-proposito es buena practica: deja constancia de que es una decision y no un olvido.

`ProdAuthenticationProvider` si valida. Y ademas, cuando el email no existe, lanza `BadCredentialsException` en vez de `UsernameNotFoundException`: asi el atacante no puede distinguir "ese mail no existe" de "la clave esta mal", y no puede usar el login para averiguar que mails estan registrados.

6.2 Momento 2: todos los demas pedidos

Una vez que tenes el token, la contraseña no viaja nunca mas. Cada pedido lleva el token y atraviesa esta cadena antes de tocar un solo controller:

Llega POST /api/v1/reservas con el header Authorization: Bearer eyJ...

El interceptor de Axios se lo pego solo, leyendo la cookie. Ningun componente tuvo que acordarse.



RateLimitFilter

Se saltea: solo se aplica a /login, /register y /forgot-password.



JWTValidationFilter: EL FILTRO MAS IMPORTANTE

Toma el token, verifica la **firma** con la clave secreta. Si esta vencido -> 401. Si el email figura en la lista negra de revocados -> 401. Si esta todo bien, saca el email y los roles y los mete en el `SecurityContextHolder`: a partir de este instante, para Spring, "vos estas logueado".



authorizeHttpRequests: la tabla de permisos

Compara la URL contra la lista de reglas, en orden. Si la regla exige ADMIN y vos sos USER: **403 Forbidden**. Si exige estar autenticado y no lo estas: **401 Unauthorized**.



Recien aca entra el controller

Y detras suyo, el service con las reglas de negocio.

401 y 403 no son lo mismo (esto lo preguntan siempre)

401 Unauthorized = "no se quien sos". Falta el token, o esta vencido, o la firma no da.

403 Forbidden = "se perfectamente quien sos, y no te corresponde". El token es valido pero el rol no alcanza.

Hay tests dedicados a distinguirlos: mira `DashboardAccessSecurityTests`, que es literalmente una **matriz de que rol puede pegarle a que endpoint**.

6.3 La tabla de permisos, tal como esta escrita

Esto es lo que dice DevSecurityConfig.java, simplificado pero fiel al orden real:

```
// 1) PUBLICO - no hace falta estar logueado.
// Va PRIMERO a proposito (ver la nota del capitulo anterior).
.requestMatchers("/api/v1/cocheras/disponibles",
                 "/forgot-password",
                 "/reset-password",
                 "/actuator/health").permitAll()

// 2) SOLO ADMIN
.requestMatchers("/users/**",
                 "/api/v1/usuarios/**",
                 "/register",
                 "/api/v1/cocheras/**").hasAuthority("ADMIN")

// 3) CUALQUIER USUARIO AUTENTICADO (los usan los dos dashboards)
.requestMatchers("/api/v1/reservas/**",
                 "/api/v1/vehiculos/**",
                 "/api/v1/visitantes/**",
                 "/login").authenticated()

// 4) TODO LO DEMAS, abierto (Swagger, recursos estaticos)
.requestMatchers("/**").permitAll()
```

6.4 Las otras tres piezas de la configuracion

- **CSRF desactivado** (.csrf(...disable)). El ataque CSRF aprovecha que el navegador manda las cookies solo; como aca la autenticacion va en un header que el navegador **no** agrega automaticamente, el ataque no aplica. Esta bien desactivarlo en una API con JWT.
- **Sesion STATELESS**. El servidor no guarda absolutamente nada entre pedidos. Toda la identidad viaja en el token. Esto es lo que permitiria correr diez copias del backend detras de un balanceador sin que nadie pierda la sesion.
- **CORS restringido a localhost**. Por defecto un navegador prohíbe que una pagina de un dominio le pegue a otro. Como el front esta en el puerto 3000 y el back en el 8080, son orígenes distintos y hace falta permiso explicito. La configuracion permite `http://localhost:*` y, clave, expone el header `Authorization`: sin ese `setExposedHeaders`, el JavaScript del login **no podria leer el token** aunque el backend se lo mandara.

6.5 La seguridad del lado del frontend

El frontend tambien tiene proteccion, pero recuerda: es **comodidad, no seguridad**. Sirve para no mostrarle a un USER un menu que igual no va a poder usar. Hay dos mecanismos, y el proyecto prefiere el primero:

Mecanismo	Donde corre	Como se usa	Fortaleza
requireAuth() (server-side)	En el servidor de Next.js, antes de mandar HTML al navegador	<code>await requireAuth(["ADMIN"])</code> en el <code>page.jsx</code>	Fuerte: el usuario nunca recibe la pagina. Sin token va a <code>/login</code> ; con token pero sin rol, a <code>/unauthorized</code> .
ProtectedRoute (client-side)	En el navegador, despues de cargar	Envolviendo el componente	Debil: el HTML ya llevo. Es el metodo viejo, se mantiene por compatibilidad.

La base de datos: como se crea, como se guarda

La base de datos de AparcAR es PostgreSQL y corre en su propio contenedor Docker. Lo interesante no es eso, sino **como se crea su estructura**, porque el proyecto usa dos mecanismos a la vez y eso genera consecuencias reales.

7.1 Las tablas que existen hoy

Tabla	Quien la crea	Columnas principales	Clase Java
visitantes	Liquibase	id, nombre, documento, telefono, email, app_user_id	Visitante.java
vehiculos	Liquibase	id, patente, tipo, visitante_id	Vehiculo.java
cocheras	Liquibase	id, numero, sector, tipo, estado	Cochera.java
reservas	Liquibase	id, fecha, visitante_id, vehiculo_id, cochera_id, estado, fecha_creacion	Reserva.java
app_users	Hibernate (ddl-auto)	id, nombre, email, password, telefono, is_active	AppUser.java
app_user_authorities	Hibernate (ddl-auto)	user_id, authority	AppUser.authorities
otp_codes	Hibernate (ddl-auto)	id, token, user_id, expires_at, used	OneTimePassword.java
databasechangelog databasechangeloglock	Liquibase	Registro interno de que migraciones ya se aplicaron	(ninguna)

7.2 Los dos mecanismos, y por que conviven

	Liquibase	Hibernate ddl-auto
Que es	Archivos versionados que describen cada cambio de la estructura	Hibernate mira tus clases @Entity y adapta la base para que coincidan
Quien decide	Vos, escribiendo el changeset a mano (o generandolo con make migrate)	Hibernate, automaticamente, sin que nadie revise
Ventaja	Reproducible, revisable, y va al repo. Todo el equipo tiene lo mismo.	Comodisimo: agregas un campo a la clase y aparece la columna
Riesgo	Hay que acordarse de escribirlo	En produccion puede modificar una base con datos reales sin control
Cuando corre	PRIMERO , al arrancar Spring	DESPUES de Liquibase

La consecuencia concreta: por que la migracion 003 no tiene clave foranea

Se queria vincular un visitante con su cuenta de login, agregando la columna `visitantes.app_user_id`. Lo natural seria declararla como **clave foranea** (FK) apuntando a `app_users`: asi la propia base impediria guardar un id que no existe.

No se puede. Porque `app_users` la crea **Hibernate**, que corre **despues** de Liquibase. Cuando la migracion 003 se ejecuta en una base nueva, esa tabla **todavia no existe** y la FK explota.

Detalle que lo vuelve un caso de estudio: **en la maquina de los desarrolladores funcionaba**, porque su base de Postgres ya tenia `app_users` creada de arranques anteriores. Fallaba solo en H2, que arranca siempre vacia. Es decir: **lo encontraron los tests**, no las pruebas a mano.

La solucion: la columna se agrega sin FK fisica, y la integridad la valida JPA en el codigo con `@OneToOne` en `Visitante.java`. Esta comentado en el propio archivo de migracion para que nadie "arregle" el supuesto olvido.

7.3 Que base se usa en cada situacion

Situacion	Base	ddl-auto	Por que
Desarrollo diario (docker compose up)	PostgreSQL en Docker	update	Datos que persisten entre reinicios, y comodidad para cambiar entidades
Correr los tests (mvn test)	H2 en memoria	create-drop	Arranca en milisegundos, siempre limpia, no ensucia nada, no necesita Docker
Produccion	PostgreSQL real	validate	Hibernate solo avisa si la base no coincide. Nunca la modifica solo.

Una cosa que se pierde y asusta: el volumen de Docker

En `docker-compose.yml` el servicio db declara volumes: - `db_volume:/var/lib/postgresql/data`.

Un **volumen** es un espacio de disco que vive **fuera** del contenedor. Sin el, cada vez que hicieras `docker compose down` perderias todos los datos, porque los contenedores son descartables por diseño. Con el volumen, los datos sobreviven.

Corolario practico: si alguna vez queres empezar de cero de verdad (por ejemplo porque la estructura quedo inconsistente), `docker compose down` **no alcanza**. Hace falta `docker compose down -v`, que tambien borra el volumen.

Docker: por que un contenedor y no un servidor normal

Esta es una de tus preguntas y merece una respuesta larga, porque es de las cosas que mas cuestan de entender al principio y que mas cambian la forma de trabajar.

8.1 Que es un contenedor, en criollo

Un **contenedor** es una caja que contiene tu programa **mas todo lo que tu programa necesita para correr**: el sistema operativo minimo, la version exacta de Java, las librerias, las variables de entorno. Esa caja se arma siguiendo una receta escrita (el `Dockerfile`) y despues corre **identica** en cualquier maquina que tenga Docker: Windows, Mac, Linux, un servidor en la nube.

No es una maquina virtual. Una VM emula una computadora entera y arranca en minutos; un contenedor comparte el nucleo del sistema operativo anfitrión y arranca en segundos, pesando una fraccion.

8.2 El problema que resuelve

"En mi maquina anda"

La frase mas costosa de la industria. Vos tenes Java 21, tu compañero tiene Java 17. Vos tenes PostgreSQL 16, el servidor de la facultad tiene la 13. Vos estas en Windows, el profesor corrige en Mac. El codigo es identico y el comportamiento no.

Docker elimina esa clase entera de problemas: la version de Java, de Maven y de Postgres estan **escritas en el repositorio**, en los `Dockerfiles`. Nadie puede tener una distinta aunque quiera.

8.3 Las siete razones concretas, en este proyecto

1. **No hace falta instalar Java ni Maven ni PostgreSQL.** Lo dice el README textualmente. Un integrante nuevo instala Docker y ya. Sin Docker tendria que instalar el JDK 21, configurar `JAVA_HOME`, instalar Maven, instalar PostgreSQL, crear la base, crear el usuario, darle permisos. Eso es media tarde y varios problemas.
2. **La version queda congelada y versionada.** `FROM eclipse-temurin:21-alpine` y `FROM postgres:16.12-alpine`. Esas lineas estan en git: son parte del codigo, se revisan en un pull request como cualquier otra cosa.
3. **La base de datos viene incluida y preconfigurada.** El `docker-compose.yml` levanta Postgres, le crea la base y el usuario con el script `db-start.sh`, y lo conecta al backend. Un comando.
4. **El orden de arranque se resuelve solo.** El servicio `server` declara `depends_on: db: condition: service_healthy` y la base tiene un healthcheck con `pg_isready`. Traduccion: "no arranques el backend hasta que la base conteste que esta lista". Sin esto, el backend arrancaria mas rapido que la base y moriria con un "Connection refused" cada vez.
5. **Borrar y empezar de cero es trivial.** Si algo queda inconsistente: `docker compose down -v` y arrancas limpio. Sin contenedores, "desinstalar bien" PostgreSQL de Windows es una aventura.
6. **No ensucia tu computadora.** Todo vive adentro de los contenedores. No hay servicios corriendo en segundo plano cuando apagas el proyecto.
7. **Lo mismo que corre en tu maquina es lo que corre en el servidor.** Desplegar deja de ser "copiar archivos y rezar" y pasa a ser "correr la misma imagen en otro lado".

8.4 Leyendo el Dockerfile del backend (y el truco del multi-stage)

```
# ----- ETAPA 1: "build". Aca se compila. -----
FROM maven:3.9.9-eclipse-temurin-21-alpine AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean install -DskipTests      # <- compila y genera el .jar

# ----- ETAPA 2: la imagen final. Aca se ejecuta. -----
FROM eclipse-temurin:21-alpine        # <- SOLO Java, sin Maven
WORKDIR /app
COPY --from=build /app/target/*.jar server.jar  # trae solo el .jar
CMD ["java", "-jar", "server.jar"]
```

Por que dos etapas: el multi-stage build

La primera imagen trae Maven y todo el JDK: pesa cientos de megas y tiene el código fuente adentro. La segunda trae solo el runtime de Java.

El `COPY --from=build` se lleva **únicamente el .jar** de la primera a la segunda. La imagen final es mucho mas chica y, sobre todo, **no contiene tu código fuente ni las herramientas de compilación**: menos superficie de ataque.

Consecuencia practica que te va a pasar: como la imagen final no tiene Maven, no puedes correr `mvn test` dentro de ese contenedor. Por eso el README propone levantar un contenedor temporal de Maven solo para los tests, montando el código:

```
docker run --rm -v "${PWD}:/app" -w /app maven:3.9.9-eclipse-temurin-21-alpine mvn test
```

8.5 Y el frontend, por que no se levanta con Docker?

Existe el Dockerfile y el `docker-compose.yml` del frontend (y `next.config.mjs` tiene `output: standalone`, que es justamente la opción pensada para contenedores). Pero para el día a día el README recomienda `npm run dev` directo, por una razón simple: **el hot reload**. Guardas un archivo y la página se actualiza sola en menos de un segundo. Dentro de un contenedor eso es mas engorroso. En producción si se usaria la imagen.

Manual de arranque: como se ejecuta todo

El orden importa. Siempre este:

1) Docker Desktop abierto 2) Backend 3) Frontend. El frontend sin backend carga la pantalla pero todos los formularios tiran error de red.

9.1 Una sola vez por maquina

Herramienta	Para que	Verificar con
Git	Clonar el repositorio	git --version
Docker Desktop	Levantar el backend y la base sin instalar Java, Maven ni Postgres	docker --version
Node.js 20 LTS (trae npm)	Correr el frontend	node --version / npm --version

Despues de instalar Docker Desktop hay que **abrirlo y dejarlo corriendo**. Si no esta activo, todos los comandos docker fallan.

9.2 Levantar el backend

```
cd aparcAR-api-back

# Windows (PowerShell):
Copy-Item .env.example .env
# Mac / Linux:
cp .env.example .env

# Build + arranque, todo junto:
docker compose up --build
```

Que hace cada parte. docker compose up levanta los dos servicios definidos en el docker-compose.yaml: db y server. La bandera --build es **el build**: obliga a Docker a recompilar la imagen del backend desde cero corriendo Maven adentro. **Siempre pasala si el codigo cambio**, porque si no Docker reutiliza una imagen vieja en cache y vas a estar probando el programa de ayer.

Vas a ver mucho log de descarga y compilacion, despues el banner ASCII de AparcAR, y finalmente Started AparcarApiApplication in X.XXX seconds. La terminal se queda mostrando logs: **eso esta bien**, significa que el servidor sigue vivo. No la cierres.

Verificacion del backend

Abri <http://localhost:8080/docs>. Si carga Swagger con la lista de endpoints, esta funcionando. Ademas podes probar cada endpoint desde ahi mismo.

9.3 Levantar el frontend

En una **terminal nueva**, dejando la del backend corriendo:

```

cd aparcAR-front

Copy-Item .env.example .env      # Windows
cp .env.example .env            # Mac / Linux

npm install                      # solo la 1ra vez o si cambio package.json
npm run dev                      # desarrollo, con recarga automatica

# Para probar el build real de produccion:
npm run build
npm start

```

El `.env` define `NEXT_PUBLIC_API_BASE_URL=http://localhost:8080`: la direccion donde el frontend espera encontrar al backend. Si el backend corre en otro puerto, se edita ahi. Verificacion: abri **`http://localhost:3000`**.

9.4 Correr los tests

```

# BACKEND - 149 tests. Usa H2 en memoria: NO necesita Docker levantado.
cd aparcAR-api-back
mvn test                                # si tenes Maven instalado

# Si NO tenes Maven, con un contenedor temporal (Windows PowerShell):
docker run --rm -v "${PWD}:/app" -w /app maven:3.9.9-eclipse-temurin-21-alpine mvn test

# Reporte de cobertura (JaCoCo) -> target/site/jacoco/index.html
mvn verify

# FRONTEND - 77 tests
cd aparcAR-front
npm test                                # corre una vez
npm run test:watch                      # se queda mirando y re-corre al guardar

```

9.5 Apagar todo, y que hacer si algo falla

Frontend: `Ctrl+C`. Backend: `Ctrl+C` y, si quieres liberar los contenedores, `docker compose down`.

Sintoma	Causa probable	Solucion
Connection refused en los logs del backend	La base no estaba lista, o Docker Desktop no esta corriendo	Verificar que Docker este abierto y reintentar
port is already allocated	Ya hay algo usando el 8080 o el 5433	Cerrar ese programa, o cambiar el puerto en <code>docker-compose.yml</code>
BUILD FAILURE durante el build	Error real de compilacion en el codigo Java	Leer el log: dice el archivo y la linea
La pagina carga pero los formularios fallan	El backend no esta corriendo, o la URL del <code>.env</code> apunta mal	Chequear <code>http://localhost:8080/docs</code> y revisar <code>NEXT_PUBLIC_API_BASE_URL</code>
npm install falla	Version de Node vieja	Necesitas Node 20 o superior
Te pide usuario y contraseña dos veces al loguearte	El bug del interceptor de Axios (ya corregido, ver capitulo 11)	Verificar que <code>app/api.jsx</code> tenga el <code>if (!config.headers.Authorization)</code>

Los tests: 226 razones para dormir tranquilo

El proyecto tiene **226 tests repartidos en 33 archivos**. Un test es código que prueba otro código: ejecuta una función con datos conocidos y verifica que el resultado sea el esperado. Si alguien rompe algo sin querer, los tests lo detectan antes de que llegue a un usuario.

	Archivos	Tests	Con que herramientas	Como se corre
Backend	23	149	JUnit 5, Mockito, MockMvc, H2	mvn test
Frontend	10	77	Vitest, React Testing Library, jsdom, axios-mock-adapter	npm test
Total	33	226		

10.1 Caja blanca y caja negra

El proyecto etiqueta explícitamente sus tests con estos dos términos. La distinción es de examen, así que va derecho:

	Caja BLANCA	Caja NEGRA
Que ve	Las tripas: llama a la clase directamente	Solo la interfaz pública: manda un pedido HTTP y mira la respuesta
Que prueba	Que una pieza concreta funciona bien por dentro	Que el sistema completo se comporta bien de punta a punta
Velocidad	Muy rápido (milisegundos, todo mockeado)	Más lento (levanta Spring y una base H2)
Si falla	Sabes exactamente que clase está mal	Sabes que algo del circuito se rompió, pero no que
Ejemplos aca	JWTGeneratorFilterTests, RateLimitFilterTests, ProdAuthenticationProviderTests, todo service/	Todo integration/: ReservaControllerTests, CocheraControllerTests, DashboardAccessSecurityTests

Por que las reglas de reserva están testeadas DOS veces

ReservaServiceTests (caja blanca, con mocks) y ReservaControllerTests (caja negra, contra base real) prueban **las mismas cinco reglas**. No es redundancia por descuido: es deliberado.

El de caja blanca corre en milisegundos y te dice con precisión quirúrgica si rompiste la lógica. El de caja negra es lento pero confirma que además **la plomería funciona**: que el JSON se deserializa bien, que la transacción se confirma, que el error se traduce al código HTTP correcto. Uno prueba el pensamiento, el otro prueba el circuito.

10.2 Los tests que más vale la pena leer

Archivo	Por que leerlo
integration/DashboardAccessSecurityTests	Es literalmente la matriz de permisos del sistema convertida en código: que rol puede pegarle a que endpoint. Si alguna vez dudas de un permiso, la respuesta está acá.

Archivo	Por que leerlo
integration/LoginFlowTests	El unico que levanta un servidor real. Leé el comentario de la clase: explica la trampa de MockMvc mejor que cualquier manual.
service/ReservaServiceTests	Las cinco reglas de negocio, una por test, con nombres en castellano. Es la especificacion del sistema.
service/CocheraServiceTests	Incluye el caso raro: al deshabilitar una cochera, sus reservas CONFIRMADAS pasan a CANCELADA automaticamente.
test/api.test.jsx (front)	Prueba los dos interceptores de Axios, incluida la regresion del bug del login doble.
test/dashboard-user/ReservasContent.test.jsx	Prueba el flujo completo de reservar por patente, y la regresion del bug de cache de vehiculos.

10.3 Que es un test de regresion

Un **test de regresion** se escribe **despues** de encontrar un bug, y lo unico que hace es verificar que ese bug puntual no vuelva. Es la practica mas rentable del testing: cada bug que se arregla queda vacunado para siempre. En AparcAR hay dos, y son el tema del proximo capitulo.

Tres bugs reales que ensenan mas que cien teorías

Estos tres problemas ocurrieron de verdad durante el desarrollo, estan documentados en el repo, y cada uno enseña algo que no se aprende leyendo un manual.

11.1 El login que pedia la contrasenia dos veces

Sintoma	Al iniciar sesion, el navegador mostraba el cartel nativo de usuario y contraseña, a veces dos veces seguidas, aunque el formulario ya se hubiera completado bien.
Causa	El interceptor de Axios en <code>app/api.jsx</code> pegaba el header <code>Authorization: Bearer <token de la cookie></code> a todos los pedidos. Pero el login manda a proposito un <code>Authorization: Basic ...</code> puesto a mano. El interceptor lo pisaba con un Bearer viejo y vencido, el backend respondia 401, y el navegador reaccionaba abriendo su propio dialogo.
Arreglo	Tres lineas: preguntar antes de escribir. <code>if (!config.headers.Authorization) { ...pegar el Bearer... }</code>
La leccion	El codigo que corre automaticamente para todo (interceptores, filtros, middleware) es comodisimo y por eso mismo peligroso: hay que preguntarse siempre que pasa en el caso excepcional . Casi siempre hay uno.

11.2 El vehiculo recién cargado que no aparecia

Sintoma	En el dashboard del usuario, alguien cargaba un vehiculo nuevo en "Mis datos" y, al bajar a "Nueva reserva" en la misma pantalla, la patente no figuraba.
Causa	El componente de reservas pedia la lista de vehiculos una sola vez, al montarse . Como las dos secciones conviven en la misma pagina, el componente nunca se volvia a montar y su lista quedaba congelada en el momento de la carga.
Arreglo	Agregar un <code>onFocus</code> al campo de patente: cada vez que el usuario hace click ahi, se vuelve a pedir la lista.
La leccion	En una interfaz, los datos que mostras son una foto del momento en que los pediste. Si algo puede haber cambiado desde entonces, tenes que decidir explicitamente cuando refrescar. Hay un test de regresion dedicado: "el foco en el campo de patente vuelve a pedir vehiculos y visitantes".

11.3 El usuario que no se podia borrar

Sintoma	Al eliminar una cuenta desde el panel de administracion, el backend fallaba con una violacion de clave foranea.
Causa	Si esa cuenta tenia un perfil de visitante vinculado, la fila de visitantes seguia apuntando a la fila de <code>app_users</code> que se estaba borrando. La base, bien, se negaba a dejar una referencia colgada.
Arreglo	En <code>UserService.deleteUser()</code> : primero desvincular el visitante (dejarle el <code>app_user_id</code> en nulo, sin borrar a la persona) y despues borrar la cuenta.
La leccion	Borrar nunca es una sola operacion: hay que pensar el orden y que pasa con todo lo que apunta a eso. Y ojo con la decision de diseño: se eligio conservar al visitante (perdio su login, sigue existiendo como persona que estaciona) en vez de borrarlo en cascada. Es una decision de negocio, no tecnica.

El patron comun de los tres

Ninguno era un error de sintaxis ni algo que el compilador pudiera detectar. Los tres eran **supuestos implicitos que resultaron falsos**: "el interceptor siempre debe poner el token", "la lista de vehiculos no cambia", "borrar una fila es borrar una fila".

Por eso los tres terminaron con un test que deja el supuesto **escrito y verificado**. Un test no es solo una red de seguridad: es documentacion que no puede quedar desactualizada, porque si se desactualiza, falla.

Chuleta final y autoevaluacion

12.1 Donde busco cada cosa

Si necesito...	Voy a...
Entender que hace el sistema en general	README.md
Saber que hace un archivo puntual	ARCHIVOS.md (los lista todos)
Saber que prueba un test	TESTS.md (los 226, caso por caso)
Agregar o cambiar una regla de negocio	service/impl/ (el archivo del concepto que toques)
Agregar un endpoint nuevo	controller/ + su DTO + el metodo en el service
Cambiar quien puede acceder a algo	security/securityConfig/DevSecurityConfig.java (y su gemelo de prod)
Agregar una columna o tabla	La entidad en entity/ + un changeset nuevo en db/changelog/
Cambiar una consulta a la base	repository/ (muchas veces alcanza con el nombre del metodo)
Agregar una pantalla	Una carpeta nueva dentro de app/ con su page.jsx
Cambiar como se llama a la API desde el front	app/api.jsx
Tocar la sesion del frontend	store/authStore.js y utils/serverAuth.js
Probar la API a mano, sin frontend	Swagger en http://localhost:8080/docs

12.2 Las diez frases que resumen todo

1. AparcAR gestiona reservas de cocheras, garantizando que nunca haya dos reservas confirmadas para la misma cochera el mismo día.
2. Son **dos programas separados** que hablan por HTTP: Next.js en el 3000 y Spring Boot en el 8080, mas PostgreSQL detras.
3. Los **endpoints no estan listados en ningun archivo**: emergen de las anotaciones @RequestMapping y @PostMapping de los controllers.
4. El backend esta en capas: **controller** (recibe) da paso a **service** (decide) da paso a **repository** (guarda). Toda la logica vive en el medio.
5. La autentificacion es **Basic una sola vez** (el login) y **JWT para siempre** (los demas pedidos). El servidor no guarda sesiones.
6. **401** es "no se quien sos"; **403** es "se quien sos y no te corresponde".
7. La estructura de la base la crean **Liquibase** (tablas de negocio) y **Hibernate** (tablas de login), en ese orden, y de ahi sale el asunto de la clave foranea ausente.
8. Docker existe para que **nadie tenga que instalar Java, Maven ni PostgreSQL**, y para que todos corran exactamente las mismas versiones.
9. La validacion del **frontend es comodidad**; la del **backend es seguridad**. Nunca al reves.
10. Hay **226 tests**. Antes de dar por terminado cualquier cambio, se corren.

12.3 Autoevaluacion: si podes contestar esto, entendiste el proyecto

#	Pregunta	Donde esta la respuesta
1	Si escribo <code>POST /api/v1/reservas</code> , que archivo Java se ejecuta primero y por que ese y no otro?	Capitulo 5
2	Por que una cochera ACCESIBLE acepta una moto pero una cochera AUTO no?	Capitulo 1, regla 3
3	Que pasa exactamente entre que hago click en Ingresar y que aterrizo en el dashboard? Nombra los tres filtros que intervienen.	Capitulo 6.1
4	Cual es la diferencia entre un 401 y un 403, y que test del proyecto los distingue?	Capitulo 6.2
5	Por que la migracion 003 no tiene clave foranea, y como se descubrio que hacia falta sacarla?	Capitulos 3.2 y 7.2
6	Que significa <code>ddl-auto</code> y que valor tiene en dev, en test y en prod? Por que distintos?	Capitulo 3.2
7	Por que un visitante y un usuario son cosas distintas? Puede una persona ser las dos?	Capitulo 1.3
8	Que le pasa a las reservas de una cochera cuando un admin la deshabilita?	Capitulo 1.4
9	Por que hay dos archivos (<code>page.jsx</code> y <code>UserManagement.jsx</code>) para una sola pantalla?	Capitulo 3.6, "use client"
10	Que gano con Docker que no tendria instalando Java y Postgres a mano? Nombra tres cosas.	Capitulo 8.3
11	Por que <code>LoginFlowTests</code> no usa <code>MockMvc</code> como todos los demas?	Capitulo 3.5
12	Si el frontend puede leer el JWT sin la clave secreta, por que el sistema sigue siendo seguro?	Capitulo 3.6
13	Donde agregarias la regla "no se puede reservar con mas de 30 dias de anticipacion"?	Capitulo 12.1 (en el service, y su validacion de formato en el DTO)
14	Por que el interceptor de Axios pregunta antes de pegar el token?	Capitulo 11.1
15	Que diferencia hay entre <code>docker compose down</code> y <code>docker compose down -v</code> ?	Capitulo 7.3

Como seguir a partir de aca

1. Levanta el proyecto y entra a <http://localhost:8080/docs>. Proba los endpoints desde ahi: es la forma mas rapida de que el capitulo 5 deje de ser teoria.
2. Abri `service/impl/ReservaService.java` y leelo entero. Son 120 lineas y es el corazon del sistema.
3. Abri `app/api.jsx`. Son 40 lineas y explican como se conectan las dos mitades.
4. Corre los tests y rompe algo a proposito: cambiala una regla en `ReservaService` y mira cual falla. Esa es la mejor forma de entender que hace cada uno.